SEC 8-K Product Announcement Analysis

Methodology Report | Crispen Chachengwa | MS in Financial Technology, UCF

Study period: January 1–December 31, 2025 | Output: 100 entries, 70 companies
Project: **https://github.com/criszw-collab/LLM-Document-Analysis**

1. Introduction: why document analysis matters

Company reports contain useful information, but it is spread across long documents, tables and attached releases. Reading every filing by hand takes time and makes comparisons difficult. Document analysis converts this unstructured text into a consistent table that can be searched, compared and used for financial research.

This project uses SEC Form 8-K filings to identify newly announced commercial products. A large language model (LLM) reads selected passages and identifies company and product entities. Python then checks the proposed records and saves them in a CSV. The aim is to extract supported announcements rather than treat every product mention as a new release.

2. Methodology and code flow

Step 1 — Set up Colab and Ollama. The notebook installs Python libraries and starts Ollama inside the Colab runtime. It checks GPU availability with `nvidia-smi`. A T4 GPU runtime is the intended execution setting; the uploaded notebook does not retain a readable GPU hardware result. The GPU accelerates model inference, while the CPU handles downloads, parsing and file operations. Google Drive stores progress across sessions.

Step 2 — Discover company filings. The SEC `company_tickers.json` file maps stock tickers to company identifiers (CIKs). The instructor's browse-EDGAR Atom endpoint finds 8-K filings. SEC submissions JSON supplements historical discovery and acceptance timestamps. BeautifulSoup parses Atom XML and filing HTML; JSON is decoded directly. Candidate companies are expanded when the current filing pool produces too few product entries.

Step 3 — Prepare relevant text. The pipeline downloads the primary 8-K and readable numbered Exhibit 99 documents, where company press releases often appear. It removes HTML markup and uses `pypdf` for readable PDFs. Text is divided into units of about 900 characters. Release terms such as *launch*, *introduce* and *initial shipment* select passages and nearby context for batches of about 9,000 characters. Documents without matching cues are logged as retrieval skips, not proven negatives.

> **Recorded processing scale.** An intermediate completed pool contained 2,215 attempted filings: 2,186 completed and 29 errors. Of these, 799 filings involved model calls and 1,387 were retrieval skips. Later continuation cells expanded the pool and reached 100 entries. The notebook does not print a final consolidated filing total, so 2,215 is an intermediate count, not the final total.

Step 4 — Extract with an LLM. Python queries Ollama using `qwen3:4b-instruct-2507-q4_K_M`, a quantized four-billion-parameter instruction model. The prompt performs ORGANIZATION and PRODUCT named entity recognition (NER), then requests structured JSON containing the product name, description, release type and source evidence. Settings include temperature 0, seed 42, an 8,192-token context and a 2,500-token extraction output limit. Explicit upcoming introductions may qualify; descriptions should preserve future or sampling status.

Step 5 — Check and save. JSON schema and Python checks require source-linked names and quotes and descriptions of 1–179 characters. Proposed positive entries receive another source-consistency check using the same model. Financial results, appointments and existing-product adoption alone should be excluded. The pipeline removes repeated normalized names for the same CIK, saves raw model responses and checkpoints, and updates a progress bar. Completed filings are skipped when the run resumes.

Step 6 — Export the table. Filings without accepted products add no product rows. Failed filings are logged separately. The final CSV is written when the pipeline reaches 100 accepted entries. Its five columns are `company_name`, `stock_name`, `filing_time`, `new_product` and `product_description`. The supplied `final_csv(1).csv` contains 100 rows across 70 companies, no blank required values and descriptions of 55–179 characters. These are formatting results, not measured extraction accuracy.

3. Limitations and challenges encountered

Computing and runtime. Earlier local CPU runs were slow and caused the computer to heat up. A Colab T4 setting reduces the load on the local machine, but model inference, network requests and multiple batches still take time. GPU allocation and session interruptions can limit a run. Checkpoints allow continuation without starting the whole dataset again. The saved outputs show several continuation sessions, so one session time should not be presented as the total project runtime.

Access and document quality. The notebook records SEC timeouts, HTTP 503 errors, image-only or unreadable exhibits, and truncated model JSON. Some filings therefore remain unprocessed or incomplete. Reading an image-based exhibit would require OCR, and keyword selection may miss an announcement written differently. Product-focused company selection also means the sample does not represent all listed companies.

Accuracy and remaining defects. The final CSV still includes mixed-language text and unfinished descriptions. At least 14 descriptions were flagged, and two SoloVPE names may refer to the same product. Simple name normalization cannot resolve every alias. The same model performs extraction and checking, so both passes can repeat an error. Precision and recall were not measured against independent labels; 100 saved rows should not be described as 100 independently verified products. Targeted source-based re-extraction is needed before claiming the output is fully checked.

Project link and source notes: Evidence for this report: the supplied executed notebook and `final_csv(1).csv`. SEC discovery endpoints used in the code: `company_tickers.json`; browse-EDGAR Atom feed; and SEC submissions JSON.

Requested GitHub file link: `github.com/criszw-collab/LLM-Document-Analysis/blob/main/filing_results_100.csv`. This file path returned 404 when checked; the project repository link on page 1 is accessible. The linked filename is not evidence that the supplied final product CSV is published there.